Compute Choice

For a startup, **Cloud Run** is the clear winner. GKE gives you full Kubernetes control but comes with always-on node costs, cluster management toil, and a steep ops curve. Cloud Run runs your containers serverlessly — it scales to zero when idle, bills per 100ms of request time, and handles TLS, health checks, and rollouts for you. Set `min-instances: 1` to eliminate cold starts on the critical path. When you genuinely outgrow it (sustained high concurrency, need for GPUs, or service mesh requirements), migrating to GKE Autopilot is a clean upgrade path.

Key reasons for choosing Cloud Run:

- Automatically scales up and down based on incoming traffic (even scales to zero)
- Cost-efficient since you only pay for actual usage
- No need to manage servers or clusters
- Easy integration with Docker-based CI/CD pipelines
- Built-in support for HTTPS and secure deployments

Why not other options:

- **GKE (Kubernetes):** Powerful but adds complexity and higher operational cost
- **Compute Engine (VMs):** Requires manual scaling, patching, and maintenance

GKE (Kubernetes) :	Powerful but complex + higher cost
Compute Engine (VMs)	Manual scaling, more operational overhead
Cloud Run :	Best balance of simplicity, cost, scalability

MongoDB Hosting

For the database layer, we are using **MongoDB Atlas** as a managed service instead of hosting MongoDB on our own virtual machines. From a practical engineering standpoint, managing a database in production involves backups, scaling, patching, and handling failures, which can quickly become time-consuming and error-prone. By using Atlas, we offload all of that operational burden and get a reliable, production-ready setup out of the box. It also integrates well with GCP and supports secure connections, making it a clean fit for our architecture. While self-hosting MongoDB on Compute Engine might look cheaper initially, it adds long-term maintenance overhead, so using a managed service is a better tradeoff for stability and developer productivity.

Self-hosting MongoDB on Compute Engine is cheaper until you factor in: replication setup, patch management, backup scripts, monitoring, and the on-call burden. Atlas M10 (~$60/mo) gives you a 3-node replica set, automated backups, and point-in-time recovery out of the box. The key is deploying Atlas in the same GCP region as your Cloud Run service and connecting via VPC peering or Private Endpoint — traffic stays on Google's backbone, you pay no egress fees, and the database is never exposed to the internet.

Option 1: MongoDB Atlas

- Fully managed MongoDB service
- Built-in backups, scaling, security
- Easy integration with GCP

Option 2: Self-hosted on VM

- Lower cost initially
- Requires maintenance, backups, scaling

Networking Design

Use a custom-mode VPC with private subnets. Cloud Run services are deployed with `--ingress internal-and-cloud-load-balancing`, meaning direct internet access is blocked, all traffic must pass through the **Cloud Load Balancer**, which gives you SSL termination, HTTP/2, and a global anycast IP. Put **Cloud Armor** in front of the LB for WAF rules and DDoS protection (the managed rule sets are free at the basic tier). Use **VPC Service Controls** to create a security perimeter around your GCP project, preventing data exfiltration even if credentials are compromised.

- Create a **VPC (Virtual Private Cloud)**
- Use **private subnets** for backend services
- Cloud Run connects via **Serverless VPC Connector**

Secure Access:

Never put secrets into container images or environment variables in plain text. Store all credentials (MongoDB URI, third-party API keys) in **Secret Manager** and grant Cloud Run access via a dedicated **service account** with only the `roles/secretmanager.secretAccessor` permission. Use **Workload Identity** bindings to avoid long-lived key files entirely.

- Use **HTTPS Load Balancer**
- Restrict access using:
    - IAM roles
    - Firewall rules
    - Private IP for DB access

Secrets Management:

- Use GCP Secret Manager
- Store:
    - MongoDB URI
    - API keys
    - Credentials

IAM (Access Control):

- Follow least privilege principle
- Use:
    - Service Accounts for Cloud Run
    - Role-based access (no hardcoded credentials)

Logging & Monitoring

This architecture uses centralized logging and monitoring to ensure full observability and production readiness. All application and request-level logs are structured (preferably in JSON format) and automatically ingested into GCP Cloud Logging, enabling efficient querying, filtering, and debugging. Standardized log fields such as timestamp, request ID, log level, and service name make it easier to trace requests and diagnose issues across components.

For monitoring, GCP Cloud Monitoring is used to collect system-level and application-level metrics, including request latency, error rates, and resource utilization. Custom dashboards provide real-time visibility into service health, while alerting policies are configured to proactively detect anomalies such as increased error rates or latency spikes. This approach ensures faster issue detection, reduced downtime, and improved system reliability under varying load conditions.

Implementation Flow :

Step 1: Application Logging (Spring Boot)

- Configure structured logging (JSON format)

- Include key fields:

  timestamp, log_level, service_name, request_id

Step 2: Log Ingestion

- Deploy service on Cloud Run

- Cloud Run automatically forwards logs → Cloud Logging

Step 3: Metrics Collection

- Enable Cloud Monitoring

- Collect default metrics:

  - request_count

  - latency

  - error_rate

  - CPU / memory usage

Step 4: Custom Metrics (Optional)

- Enable Spring Boot Actuator (/actuator/metrics)

- Export additional application metrics

Step 5: Dashboard Creation

- Create dashboards in Cloud Monitoring:

  - Latency graph

  - Error rate graph

  - Traffic (request volume)

Step 6: Alerting Policies

- Define thresholds:

  IF error_rate > threshold → trigger alert

  IF latency > threshold → trigger alert

Step 7: Notifications

- Configure alert channels:

  - Email

  - Slack / PagerDuty

Step 8: Debugging Workflow

- Use Cloud Logging:

  filter by request_id

  trace logs across requests

CI/CD:
Store container images in Artifact Registry (not Docker Hub — keeps everything inside GCP's IAM boundary). Wire Cloud Build to your Git repo: on merge to main, build → push → deploy to Cloud Run with `--no-traffic` first, then shift traffic after smoke tests pass. This gives you zero-downtime deployments with instant rollback.